

Module «Data Science»

Week 05



mario.gellrich@zhaw.ch

Dr. Mario Gellrich

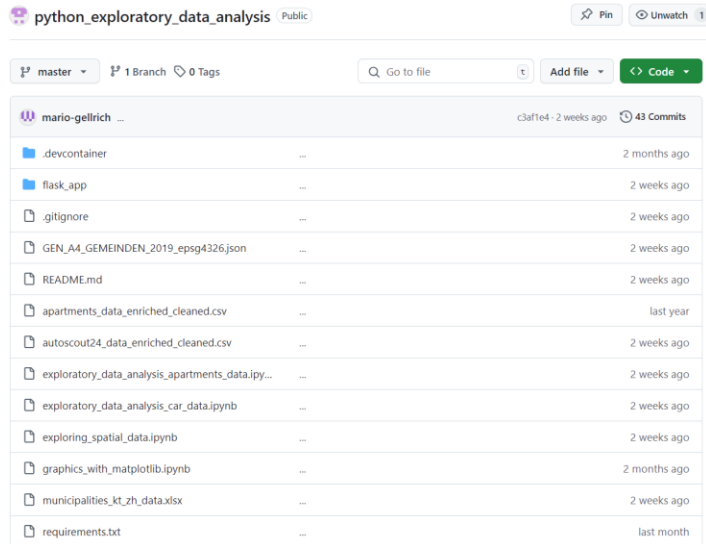
Content of this lecture

Table of contents:

- Exploratory Data Analysis (EDA)
 - Introduction to exploratory data analysis (EDA)
 - Non-graphical exploratory data analysis methods
 - Graphical exploratory data analysis methods
- Python Data Visualization Libraries for EDA
- Automated and AI-assisted EDA

Software used in this course

GitHub repository with course material for Python exercises



[Open the `python\_exploratory\_data\_analysis` repository on GitHub](#)

Tasks:

- Fork the repository into your own GitHub account
- Create a new Codespaces environment
- Read the explanations in the README section.

Introduction to Exploratory Data Analysis (EDA)

What is exploratory data analysis (EDA)?

Systematics of EDA methods

Why graphical EDA matters: Anscombe's quartet

What is exploratory data analysis (EDA)?

*“**Exploratory data analysis (EDA)** is a critical first step in analyzing the data [...]. Loosely speaking, any method of looking at data that does not include formal statistical modeling and inference falls under the term exploratory data analysis.”*

Seltman, H.J. (2018). Experimental Design and Analysis, Chapter 4: Exploratory Data Analysis – the term goes back to Tukey, J.W. (1977). Exploratory Data Analysis. Addison-Wesley.

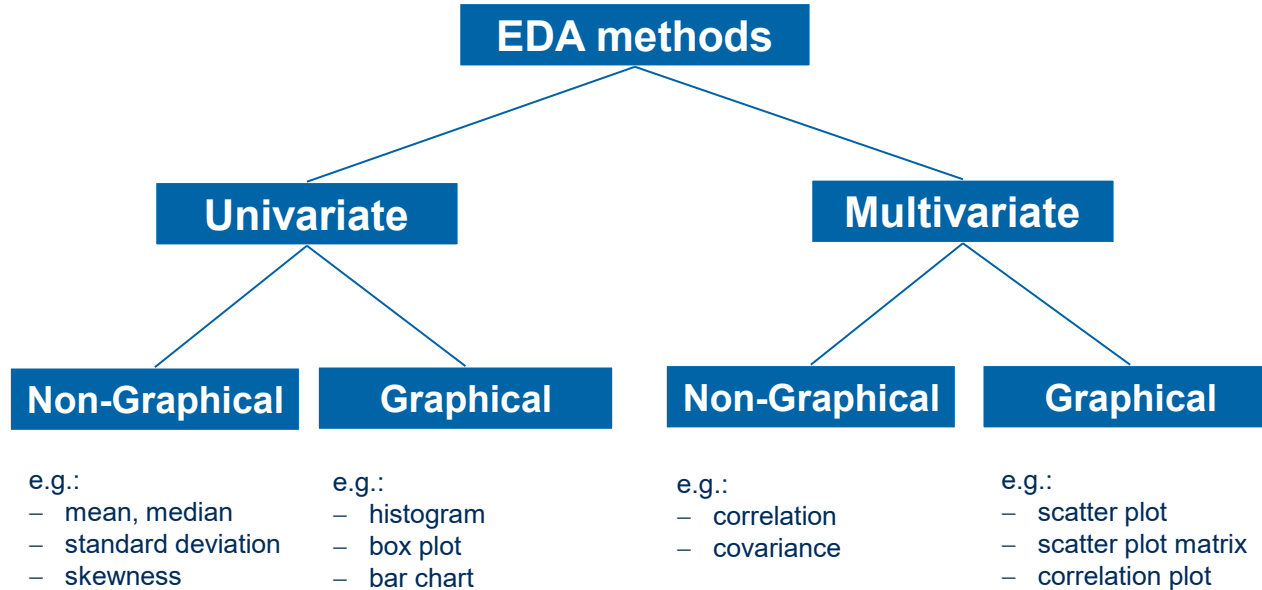
Objectives of exploratory data analysis (EDA)

The objectives of EDA are to:

- Enable unexpected discoveries in the data
- Suggest hypotheses about the causes of observed phenomena
- Assess assumptions on which statistical inference will be based
- Support the selection of appropriate statistical tools and techniques
- Provide a basis for further data collection through surveys or experiments
- Check data quality (missing values, duplicates, implausible values) before any modeling

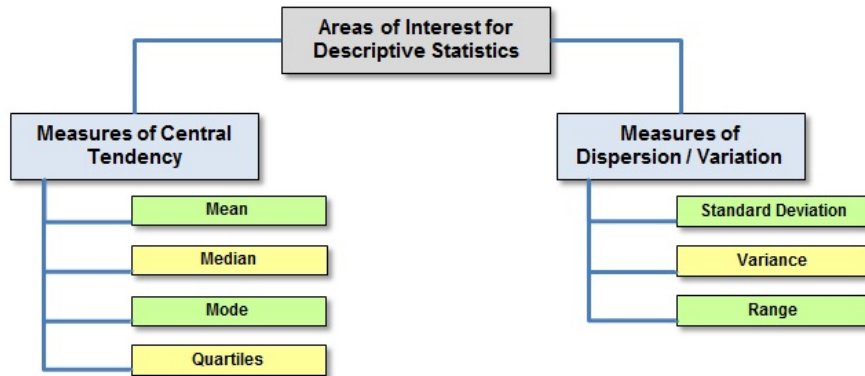
See also: Seltman, H.J. (2018). Experimental Design and Analysis, Chapter 4: Exploratory Data Analysis

Systematics of EDA methods: overview



According to: Seltman, H.J. (2018). Experimental Design and Analysis, Chapter 4: Exploratory Data Analysis

Systematics of EDA methods: univariate non-graphical



Univariate non-graphical EDA methods are concerned with understanding the distribution of single variables. This also involves outlier detection. For numerical data, statistical measures like the mean, median and standard deviation are used. For categorical data, frequencies per category are observed.

Image credit: https://www.sixsigma-institute.org/What_Is_Descriptive_Statistics.php

Systematics of EDA methods: multivariate non-graphical



	Cake	Ice	Donut	Total
Female	4	3	6	13
Male	5	7	9	21
Total	9	10	15	34

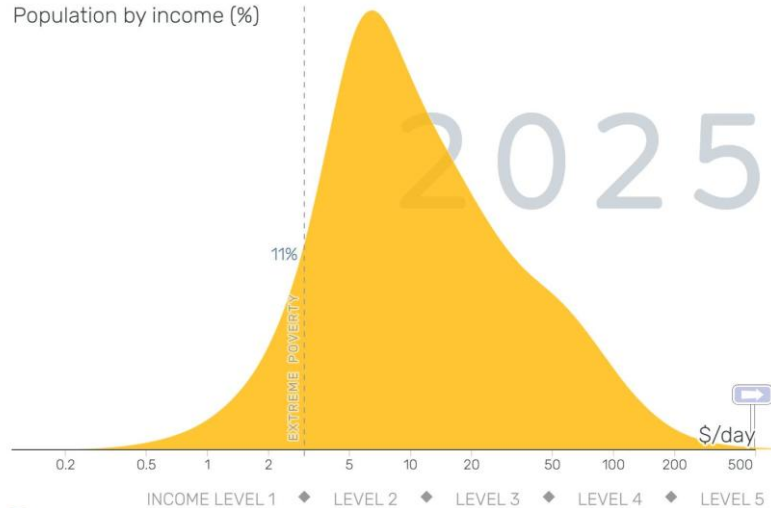
Example contingency table

Multivariate non-graphical EDA

methods generally show the relationship between two or more variables in the form of either **contingency tables** (also referred to as cross-tables) or statistics.

Image credit: <https://datatab.net/tutorial/cross-table>

Systematics of EDA methods: univariate graphical



World population by income (US\$/day, log scale), 2025.

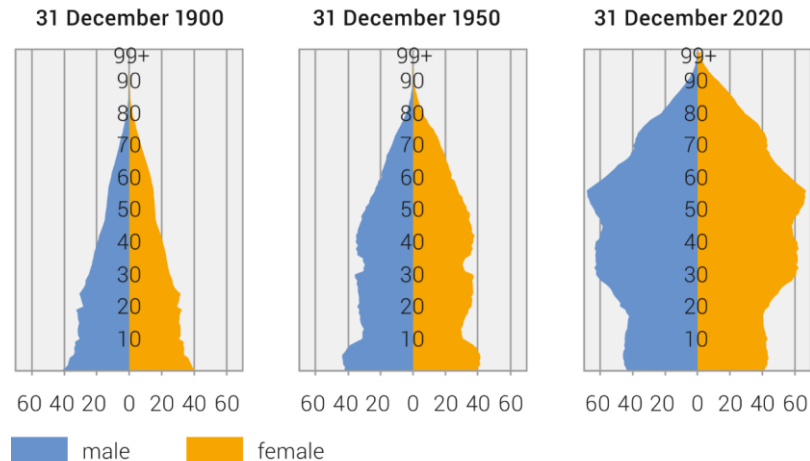
Showing a free visualization from GAPMINDER.ORG, CC-BY license (gapminder.org/tools, «Income mountains»)

Univariate graphical EDA methods are used if we are focusing on a single variable and want to complement the statistics from the non-graphical EDA methods by a graphical analysis. An example is the distribution of the daily income of the world population in 2025. Income is strongly right-skewed, which is why it is shown on a logarithmic x-axis.

Systematics of EDA methods: multivariate graphical

Population pyramid by age and sex

Number of people in 1000



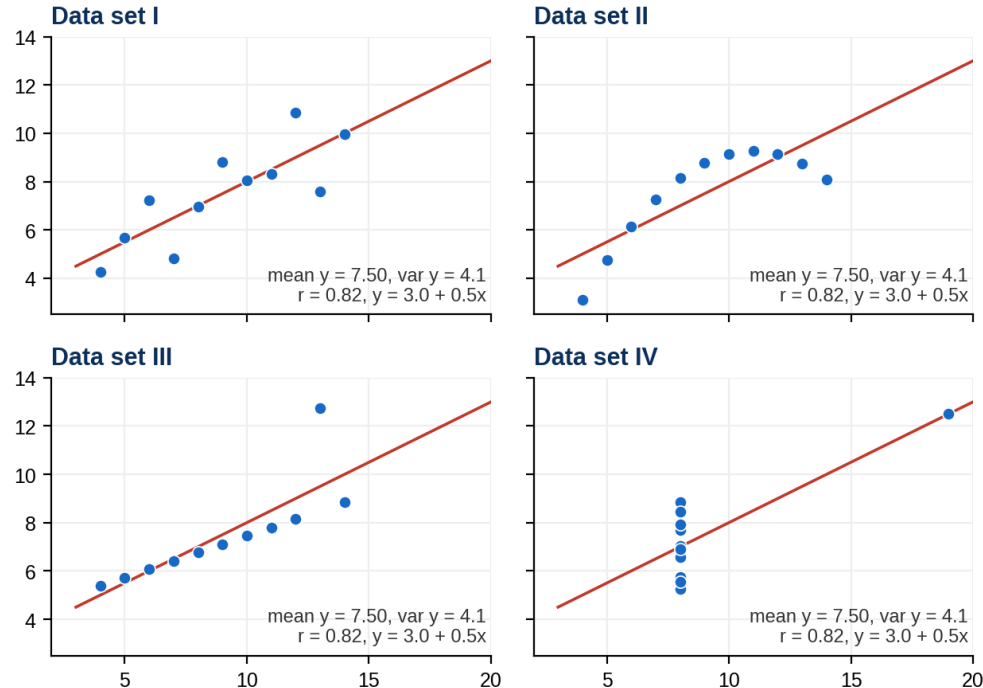
Sources: FSO – FPC, STATPOP

© FSO 2021

Image credit: <https://www.bfs.admin.ch>

Multivariate graphical EDA methods are concerned with analyzing the relationship between two or more variables graphically. An example is the age pyramid showing the age distribution by sex for different years.

Why graphical EDA matters: Anscombe's quartet



Anscombe's quartet consists of four data sets with (almost) identical means, variances, correlations and regression lines – yet completely different patterns.

Summary statistics alone can be misleading: always complement non-graphical with graphical EDA methods.

A modern variant is the «Datasaurus Dozen» (Matejka & Fitzmaurice, 2017).

Data: Anscombe, F.J. (1973). The American Statistician, 27(1), 17–21. Plot created with matplotlib.

Non-graphical exploratory data analysis (EDA) methods

Univariate non-graphical EDA methods

- Measures of central tendency
- Measures of dispersion
- Skewness and kurtosis

Multivariate non-graphical EDA methods

- Contingency tables
- Correlation & covariance

Univariate non-graphical EDA methods: Mean, median and mode

Statistical measures

$$\text{Mean } (\bar{x}) = \frac{\sum x}{N}$$

$$\text{Median} = \left(\frac{n+1}{2}\right)^{\text{th}} \text{ observation} \rightarrow (n \text{ is odd})$$

$$\text{Median} = \frac{\left(\frac{n}{2}\right)^{\text{th}} \text{ observation} + \left(\frac{n}{2} + 1\right)^{\text{th}} \text{ observation}}{2} \rightarrow (n \text{ is even})$$

$$\text{Mode} = l + \left(\frac{f_1 - f_0}{2f_1 - f_0 - f_2}\right) \times h \rightarrow \begin{array}{l} \text{(the most frequent value;} \\ \text{formula for grouped data)} \end{array}$$

Image credit: <https://www.vedantu.com/maths/statistics-formula>

```
df = pd.DataFrame({'var': [1,2,3,4,5,5,42]})
```

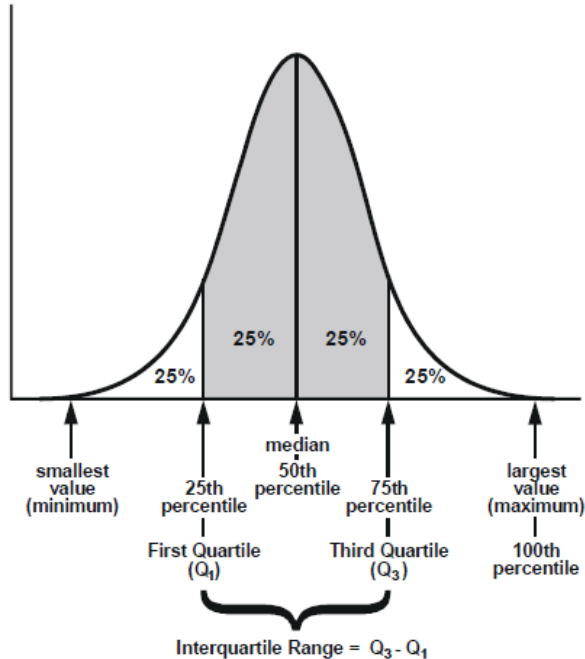
```
df.mean()  
8.857143
```

```
df.median()  
4.0
```

```
df.mode()  
5
```

Univariate non-graphical EDA methods: Quantiles

Statistical measures



In probability theory and statistics, **quantiles** are cut points dividing the range of a probability distribution or observations into continuous intervals. Common quantiles have special names, such as **quartiles** (4 groups), **deciles** (10 groups), and **percentiles** (100 groups).

```
df = pd.DataFrame({'var': [-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6]})
df.quantile([0.05, 0.50, 0.95])
0.05 -5.4
0.50 0.0
0.95 5.4
```

Univariate non-graphical EDA methods: Quantiles example

Short recap of statistical measures

Quantiles

```
1 df.quantile(q=[0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95]).round(2)
```

	bfs_number	rooms	area	luxurious	price	price_per_m2	pop_dens	frg_pct
0.05	38.0	1.0	34.0	0.0	1317.25	17.59	351.91	13.93
0.10	55.0	1.5	49.0	0.0	1480.90	19.18	525.66	18.20
0.25	115.5	2.5	65.0	0.0	1721.25	21.75	745.77	23.01
0.50	224.5	3.5	82.0	0.0	2090.00	25.47	1486.62	25.99
0.75	261.0	4.5	100.0	0.0	2600.00	32.01	3759.21	32.46
0.90	261.0	4.5	123.0	0.0	3350.00	44.96	4778.99	32.46
0.95	293.0	5.0	140.0	1.0	4155.00	53.67	4778.99	37.51

Example from the exercises showing **quantiles** for selected numerical variables. Note that $q=0.50$ equals the **median**, and that $q=0.25$ and $q=0.75$ equal the **lower** and **upper quartile**, respectively.

Univariate non-graphical EDA methods: Measures of dispersion

Statistical measures

$$\text{Variance}(\sigma^2) = \frac{\sum (x_i - \mu)^2}{N}$$
$$\text{Standard deviation}(\sigma) = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

} for the population

$$\text{Sample variance} (s^2) = \frac{\sum (x_i - \bar{x})^2}{n - 1}$$
$$\text{Sample standard deviation} (s) = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}}$$

} for the sample

Image credit:

<https://www.vedantu.com/maths/statistics-formula>

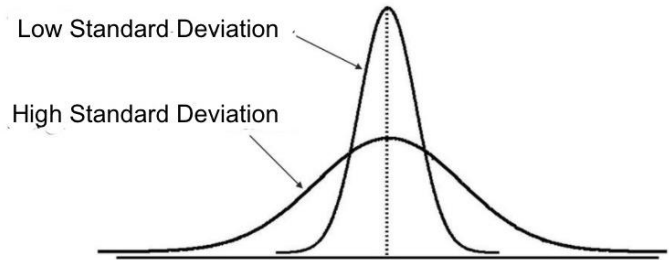
<https://careerfoundry.com>

```
df = pd.DataFrame({'var': [1,2,3,4,5,5,42]})
```

Sample variance and sample standard deviation

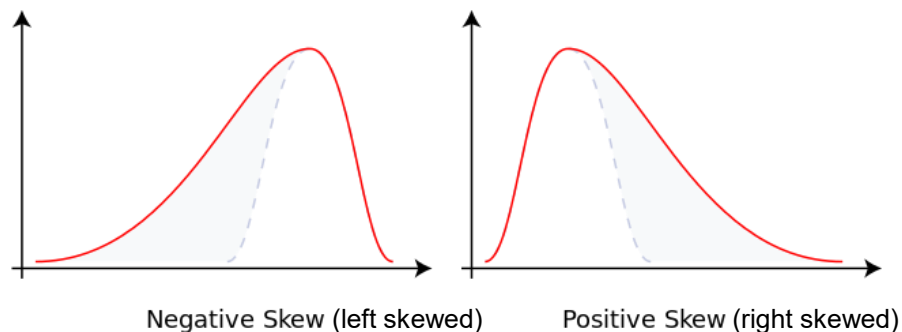
```
df.var()  
215.809524
```

```
df.std()  
14.690457
```



Univariate non-graphical EDA methods: Skewness

Statistical measures



```
df2 = pd.DataFrame({'var': [-42,-5,-5,-4,-3,-2,-1]})  
df2.skew()  
-2.58832
```

```
df = pd.DataFrame({'var': [1,2,3,4,5,5,42]})  
df.skew()  
2.58832
```

In probability theory and statistics, **skewness** is a measure of the asymmetry of the probability distribution of a variable about its mean. The skewness value can be positive (= right skewed), zero, negative (= left skewed), or undefined.

Univariate non-graphical EDA methods: Skewness example

Summary statistics of numeric variables

```
1 df.describe()
```

	bfs_number	rooms	area	luxurious	price	price_per_m2	pop_dens	frg_pct
count	870.000000	870.000000	870.000000	870.000000	870.000000	870.000000	870.000000	870.000000
mean	185.817241	3.327586	85.202299	0.070115	2296.165517	29.229540	2097.659941	26.860325
std	83.961852	1.103733	33.059531	0.255488	931.786659	12.544119	1647.377220	7.284069
min	2.000000	1.000000	12.000000	0.000000	16.000000	0.120000	82.310984	7.547170
25%	115.500000	2.500000	65.000000	0.000000	1721.250000	21.750000	745.767110	23.009684
50%	224.500000	3.500000	82.000000	0.000000	2090.000000	25.470000	1486.615110	25.994398
75%	261.000000	4.500000	100.000000	0.000000	2600.000000	32.007500	3759.212880	32.458468
max	298.000000	9.000000	300.000000	1.000000	8900.000000	126.330000	4778.994655	46.226483

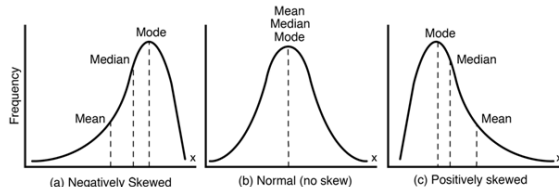
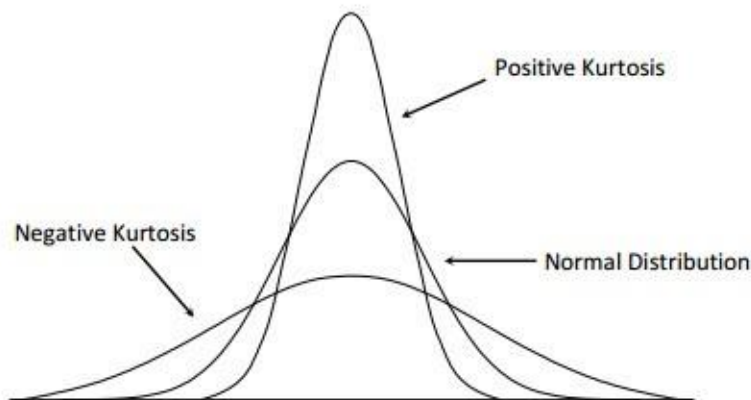


Image credit: <https://www.researchgate.net>

The **skewness** of a variable can be assessed by comparing the mean of a variable with its median (rule of thumb). In the example on the left, the mean is higher than the median showing that the distribution has positive skewness.

Univariate non-graphical EDA methods: Kurtosis

Statistical measures



```
df = pd.DataFrame({'var': [-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6]})  
df.kurtosis()  
-1.2
```

In probability theory and statistics, **kurtosis** is a measure of the "tailedness" of the probability distribution of a variable. Like skewness, kurtosis describes the shape of a probability distribution. It mainly reflects the tails, not the height of the peak.

Note: pandas returns the excess kurtosis (Fisher's definition, normal distribution = 0).

Multivariate non-graphical EDA methods: Contingency tables

Statistical measures

		sex					
		Male		Female		Total	
		Count	Column Valid N %	Count	Column Valid N %	Count	Column Valid N %
favourite means of transportation	Car	21	43,8%	17	32,7%	38	38,0%
	Plane	18	37,5%	22	42,3%	40	40,0%
	Coach	3	6,3%	9	17,3%	12	12,0%
	Train	6	12,5%	4	7,7%	10	10,0%
	Total	48	100,0%	52	100,0%	100	100,0%

Image credit: <https://peterstatistics.com>

A **contingency table** is a type of table in a matrix format that displays the frequency distribution of variables.

Multivariate non-graphical EDA methods: Contingency table example

Statistical measures

Cross-tabulation

```
1 pd.crosstab(df['luxurious'], df['rooms'])
```

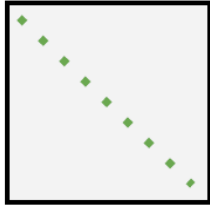
rooms	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0	5.5	6.0	6.5	9.0
luxurious													
0	39	38	34	137	64	243	50	163	12	26	1	1	1
1	6	7	2	12	0	19	0	11	1	3	0	0	0

Example **contingency table** from the exercises showing the number of apartments per room size and category (0 or 1) of variable «luxurious».

Multivariate non-graphical EDA methods: Covariance

Statistical measures

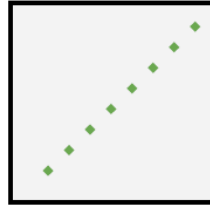
COVARIANCE



Large Negative
Covariance



Nearly Zero
Covariance



Large Positive
Covariance

$$\text{cov}(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n - 1}$$

Sample covariance

Covariance is a measure of the joint variability of two variables. It can take any value between $-\infty$ and $+\infty$, where a negative value represents a negative relationship whereas a positive value represents a positive relationship. It is used for the linear relationship between variables. It gives the direction of the relationship between variables.

Image credit: <https://www.geeksforgeeks.org/mathematics-covariance-and-correlation>

Multivariate non-graphical EDA methods: Covariance matrix

Statistical measures

Covariance matrix

```
1 cov = df.cov()  
2 cov
```

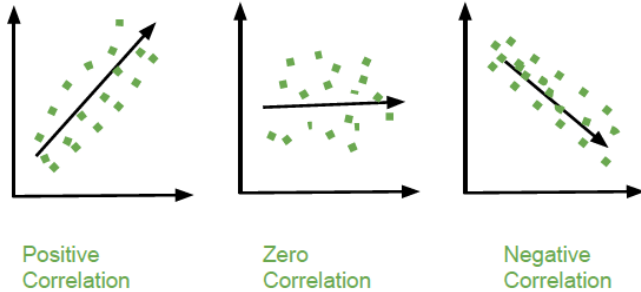
	bfs_number	rooms	area	luxurious	price	price_per_m2	pop_dens	frg_pct
bfs_number	7049.592568	-14.153526	-256.470465	1.183140	18580.540066	352.377189	6.936529e+04	135.607147
rooms	-14.153526	1.218225	24.959545	-0.017241	368.333519	-6.035304	-3.805841e+02	-1.119213
area	-256.470465	24.959545	1092.932561	1.240115	16794.857156	-194.480413	-7.357821e+03	-29.815350
luxurious	1.183140	-0.017241	1.240115	0.065274	44.261109	0.028663	5.148939e+00	-0.004963
price	18580.540066	368.333519	16794.857156	44.261109	868226.377636	3880.858971	6.034625e+05	1500.037394
price_per_m2	352.377189	-6.035304	-194.480413	0.028663	3880.858971	157.354924	1.142510e+04	31.206409
pop_dens	69365.286461	-380.584057	-7357.820609	5.148939	603462.457143	11425.102490	2.713852e+06	7715.924823
frg_pct	135.607147	-1.119213	-29.815350	-0.004963	1500.037394	31.206409	7.715925e+03	53.057665

A **covariance matrix** is a table showing the covariance of sets of variables. On the left is an example covariance matrix showing the covariances of variables in the apartment data set.

Multivariate non-graphical EDA methods: Correlation

Statistical measures

CORRELATION



$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Pearson correlation coefficient

Correlation is any statistical relationship, whether causal or not, between two variables. Correlation takes values between -1 and +1, where values close to +1 represent strong positive correlation and values close to -1 represent strong negative correlation.

Image credit: <https://www.geeksforgeeks.org/mathematics-covariance-and-correlation>

Multivariate non-graphical EDA methods: Correlation matrix

Statistical measures

Correlation matrix

```
1 corr = df.corr()  
2 corr
```

	bfs_number	rooms	area	luxurious	price	price_per_m2	pop_dens	frg_pct
bfs_number	1.000000	-0.152728	-0.092397	0.055155	0.237498	0.334569	0.501496	0.221731
rooms	-0.152728	1.000000	0.684032	-0.061142	0.358147	-0.435908	-0.209312	-0.139211
area	-0.092397	0.684032	1.000000	0.146823	0.545209	-0.468963	-0.135101	-0.123814
luxurious	0.055155	-0.061142	0.146823	1.000000	0.185924	0.008944	0.012234	-0.002667
price	0.237498	0.358147	0.545209	0.185924	1.000000	0.332025	0.393134	0.221010
price_per_m2	0.334569	-0.435908	-0.468963	0.008944	0.332025	1.000000	0.552875	0.341531
pop_dens	0.501496	-0.209312	-0.135101	0.012234	0.393134	0.552875	1.000000	0.643015
frg_pct	0.221731	-0.139211	-0.123814	-0.002667	0.221010	0.341531	0.643015	1.000000

A **correlation matrix** is a table showing correlation coefficients (like Pearson correlation coefficient) between sets of variables. On the left is an example correlation matrix from the exercises showing correlations of variables in the apartment data set.

Tip: `df.corr(method='spearman', numeric_only=True)` computes rank correlations and skips non-numeric columns.

Graphical exploratory data analysis methods

Univariate graphical EDA methods

- Line chart
- Box plot
- Histogram
- Quantile-Quantile (QQ) plot
- Bar chart
- Pie chart

Multivariate graphical EDA methods

- Scatter plot
- Bubble plot
- Scatter plot matrix
- Hexagonal binning plot
- Heat map

Univariate graphical EDA methods: Line chart

Microsoft

368,15 €

↑ 192,28 % +242,19 5 J.

4. Sept., 11:24:59 GMT+2 · EUR · ETR · Haftungsausschluss

1 T. 5 T. 1 M. 6 M. YTD 1 J. 5 J. MAX

Wichtige Ereignisse >

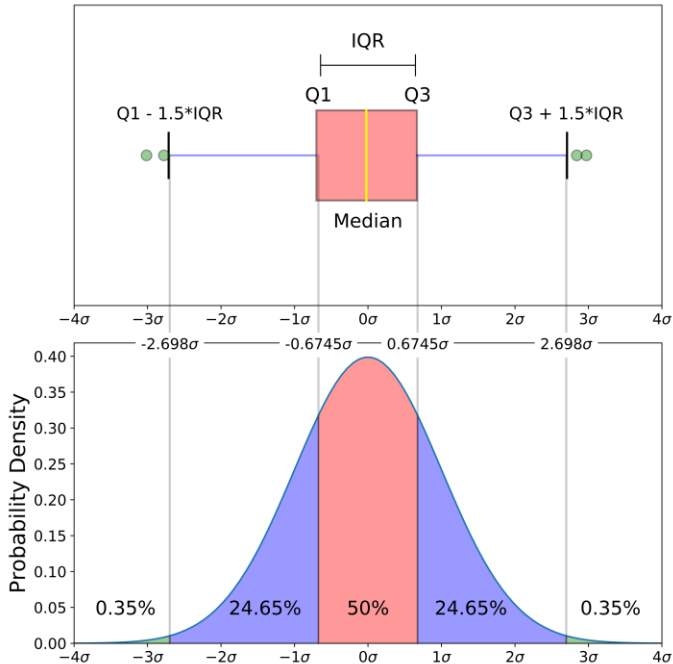


Image credit: Microsoft share price on Google Finance (5 years)

A **line chart** is a type of chart which displays information as a series of data points called 'markers' connected by straight line segments. It is often used to graphically explore time series data (one variable over time).

Univariate graphical EDA methods: Box plot

Box plot on a normal distribution



A **box plot** is a method for graphically demonstrating the locality, spread and skewness of groups of numerical data through their quartiles. In addition to the box, there can be lines (**whiskers**) indicating variability outside the upper and lower quartiles. Therefore, the plot is also termed the box-and-whisker plot.

$IQR = \text{interquartile range}$

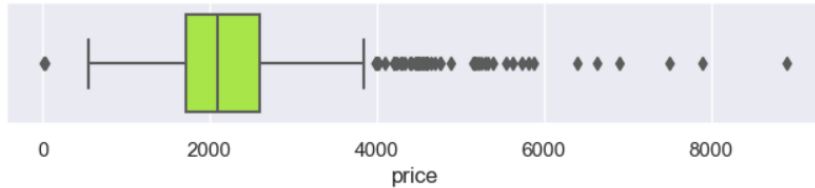
Image credit: <https://www.kdnuggets.com/2019/11/understanding-boxplots.html>

Univariate graphical EDA methods: Box plot example

Boxplot (seaborn)

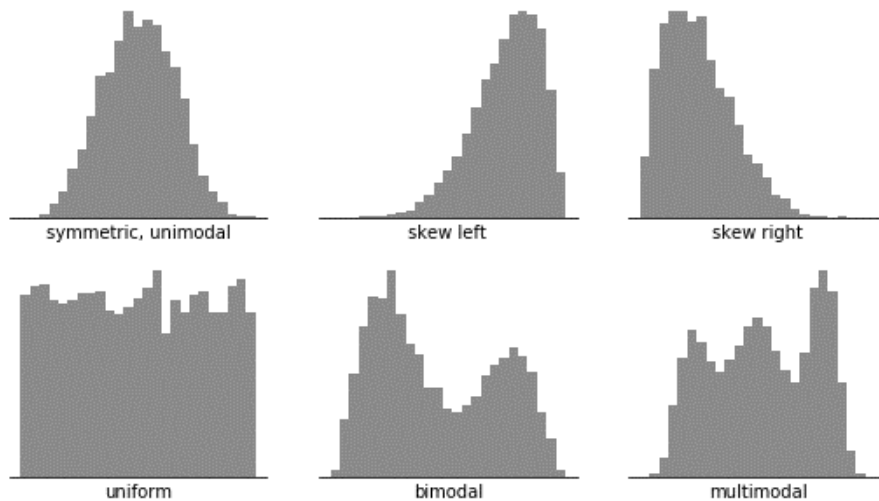
```
1 plt.figure(figsize=(8,1.2))
2 plt.ticklabel_format(style='plain')
3 sns.boxplot(x=df['price'], color="greenyellow")
```

<AxesSubplot:xlabel='price'>



Example **box plot** from the exercises with the distribution of apartment prices. It shows the median (line in box) and the price range in which the middle 50% of apartments are located (box). There are also some outliers at the lower and upper end of the price range.

Univariate graphical EDA methods: Histogram

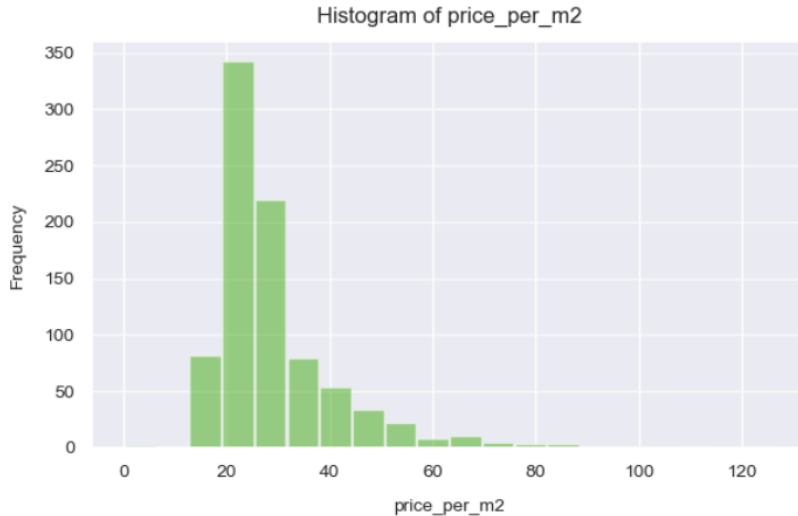


Different distributions of variables shown by histograms

A **histogram** is a graphical representation of the frequency distribution of numerical data. It requires the division of the data into classes (**bins**). Directly adjacent rectangles of the width of the respective class are drawn, the area of which represents the class frequencies. The height of each rectangle then represents the frequency density, i.e. the frequency divided by the width of the corresponding class.

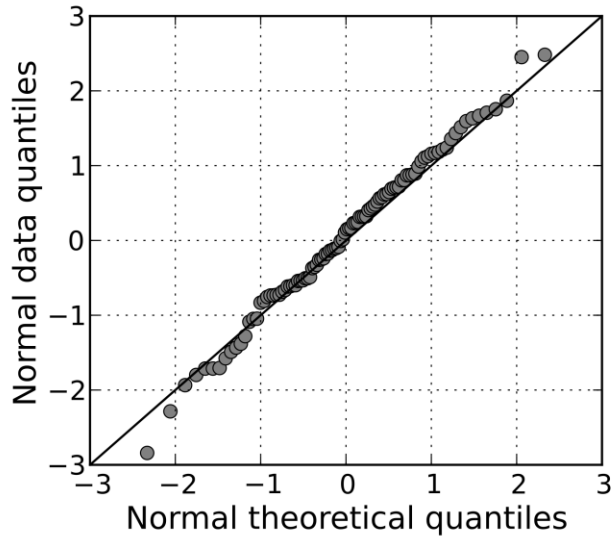
Image credit: <https://chartio.com/learn/charts/histogram-complete-guide/>

Univariate graphical EDA methods: Histogram example



Example [histogram](#) from the exercises showing the distribution of apartment prices per m². The distribution is clearly right-skewed, i.e. most apartments have a moderate price per m², while a few are very expensive (long tail to the right).

Univariate graphical EDA methods: Quantile-Quantile (QQ) plot

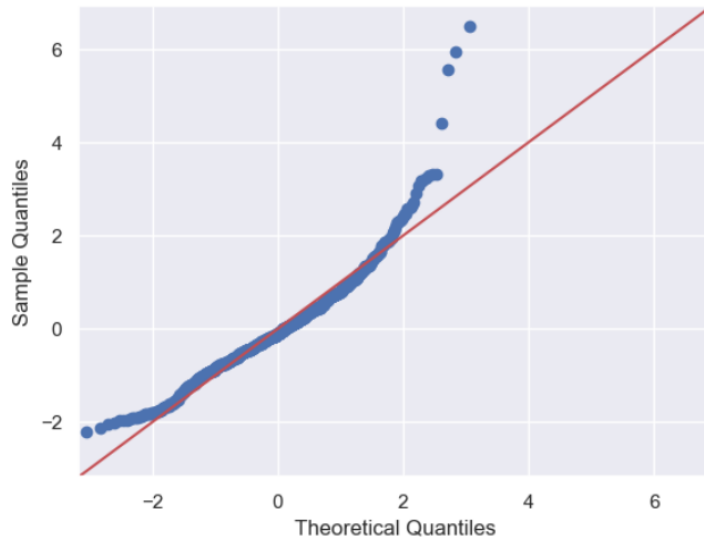


A normal QQ plot comparing randomly generated, independent standard normal data on the vertical axis to a standard normal population on the horizontal axis. The linearity of the points suggests that the data are normally distributed.

A **QQ plot (quantile-quantile plot)** is a probability plot, a graphical method for comparing two probability distributions by plotting their quantiles against each other. A point (x, y) on the plot corresponds to one of the quantiles of the second distribution (y-coordinate) plotted against the same quantile of the first distribution (x-coordinate).

Image credit: Wikipedia

Univariate graphical EDA methods: QQ plot example



Example [QQ plot](#) from the exercises showing the distribution of apartment areas. It indicates that the data are not normally distributed and that there are some outliers (very large apartment areas).

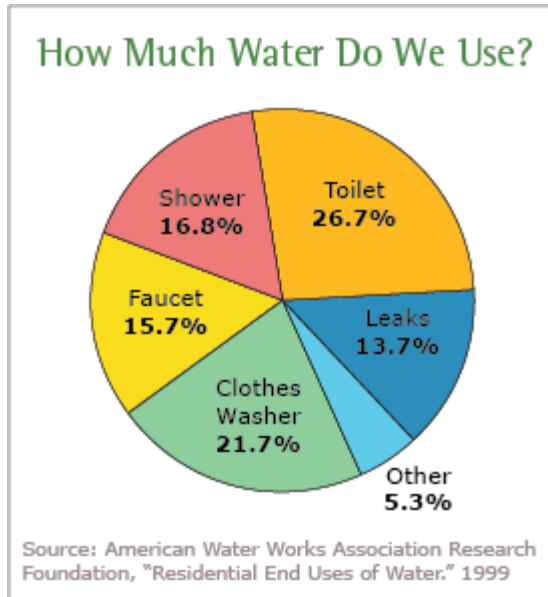
Univariate graphical EDA methods: Bar chart



A **bar chart** is a chart or graph that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent. The bars can be plotted vertically or horizontally.

Image credit: <https://www.yourteaminindia.com/blog/software-developer-hourly-rates>

Univariate graphical EDA methods: Pie chart

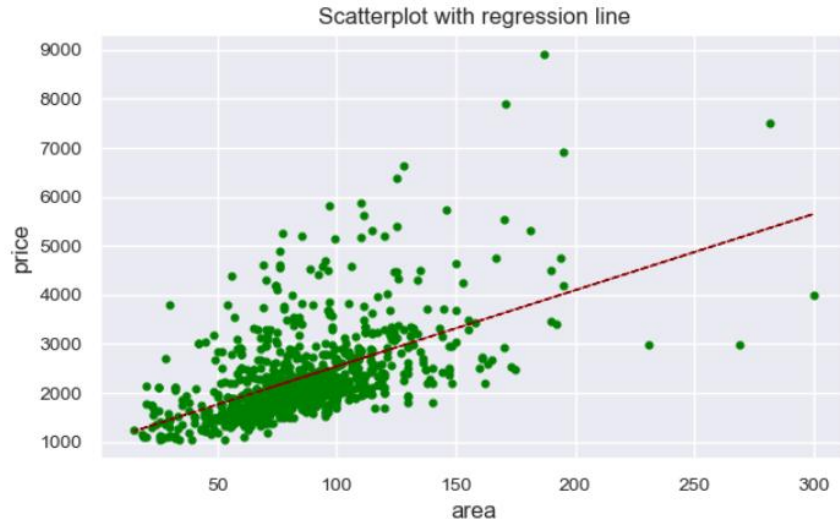


A **pie chart** is a circular statistical graphic, which is divided into slices to illustrate numerical proportion. In a pie chart, the arc length of each slice (and consequently its central angle and area) is proportional to the quantity it represents.

Use with care: angles and areas are harder to compare than lengths. For more than a few categories, a bar chart is usually the better choice (Wilke, 2019).

Data source: American Water Works Association Research Foundation (1999): Residential End Uses of Water

Multivariate graphical EDA methods: Scatter plot



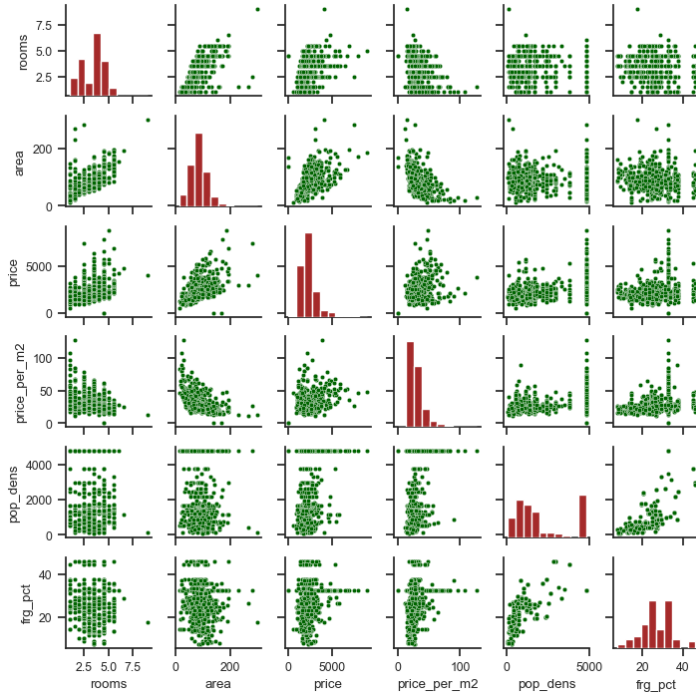
A **scatter plot** is a graphical representation of observed pairs of values. These pairs of values are plotted in a Cartesian coordinate system, resulting in a scatter plot. Left: scatter plot from the exercises with apartment prices and areas. It shows a positive relationship between price and area, i.e. the larger the area, the higher the price.

Multivariate graphical EDA methods: Bubble plot



A **bubble plot** extends a standard scatter plot by adding further dimensions. In the example on the left, the size of dots (bubbles) represents the number of rooms, and the colors represent the population density of municipalities. It shows that the most expensive apartments can be found in densely populated areas (like cities).

Multivariate graphical EDA methods: Scatter plot matrix



A **scatter plot matrix** is a grid (or matrix) of scatter plots used to visualize bivariate relationships between combinations of variables. Each scatter plot in the matrix visualizes the relationship between a pair of variables, allowing many relationships to be explored in one chart. The example on the left shows the relationships between numerical data in the apartment data set.

Multivariate graphical EDA methods: Hexagonal binning plot



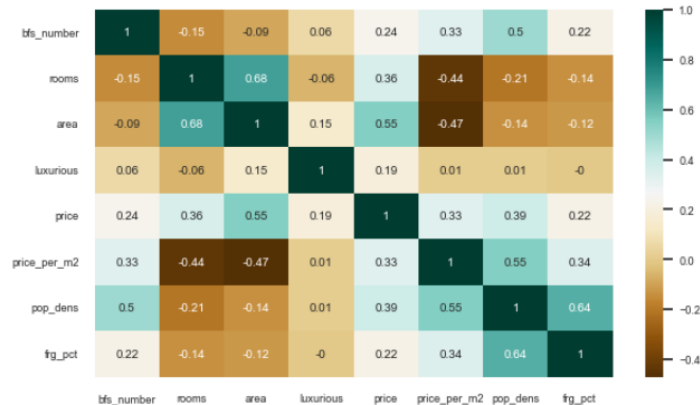
A **hexagonal binning plot** is created by covering the data range with a regular array of hexagons and coloring each hexagon according to the number of observations it covers. Hexagonal binning plots are good for visualizing large data sets for which a scatter plot would suffer from overplotting.

Multivariate graphical EDA methods: Correlation heat map

Heatmap (seaborn) based on correlation matrix

```
1 # Set font size for plot
2 sns.set(font_scale=0.6)
3
4 # Create correlation matrix as the basis for the heatmap
5 plt.figure(figsize=(7,4))
6 corr = df.corr().round(2)
7
8 # Plot heatmap
9 sns.heatmap(corr,
10             cmap="BrBG",
11             annot=True)
```

<AxesSubplot:>



A **correlation heat map** is a graphical representation of a correlation matrix representing the correlation between different variables. The correlation coefficient can take any value from -1 to 1. Correlation between two variables does not necessarily imply a causal relationship.

Python Data Visualization Libraries for EDA

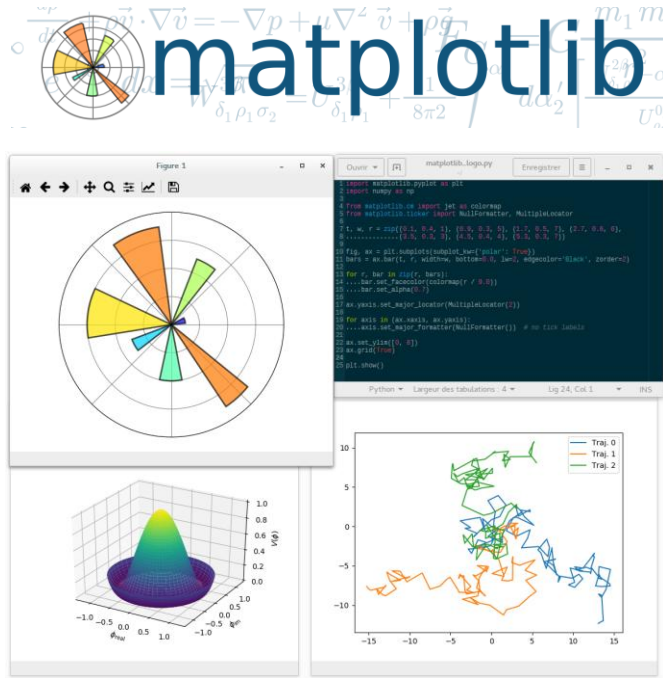
Matplotlib

Seaborn

Interactive Graphics & Web Applications

- d3blocks
- Python & Flask / FastAPI
- Plotly & Dash
- Leaflet
- Streamlit

Matplotlib: introduction



matplotlib is a plotting library for the Python programming language. **matplotlib.pyplot** is a collection of functions. Each pyplot function makes a change to a figure, e.g. creates a figure, creates a plotting area in a figure, plots lines in a plotting area, decorates the plot with labels, etc.

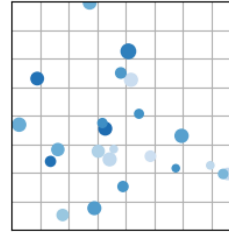
Tip: for anything beyond quick plots, use the object-oriented interface: `fig, ax = plt.subplots(); ax.hist(...)`

Matplotlib: basic plot methods

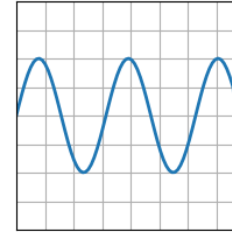
Overview of basic plot methods in matplotlib.pyplot (plt) (object-oriented: ax.scatter(), ax.plot(), ...)

- Scatter plot `plt.scatter()`
- Line chart `plt.plot()`
- Bar chart `plt.bar()` or `plt.barh()` for horizontal
- Histogram `plt.hist()`
- Box plot `plt.boxplot()`
- Pie chart `plt.pie()`

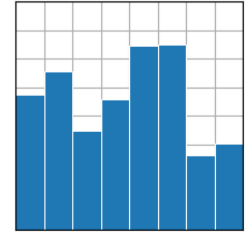
`plt.scatter()`



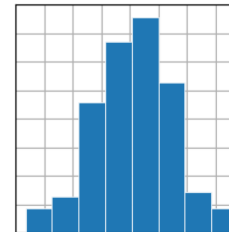
`plt.plot()`



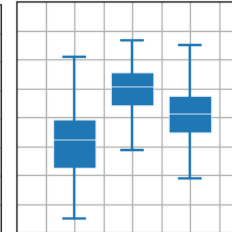
`plt.bar()`



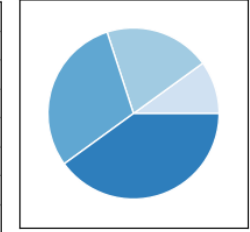
`plt.hist()`



`plt.boxplot()`



`plt.pie()`



Seaborn

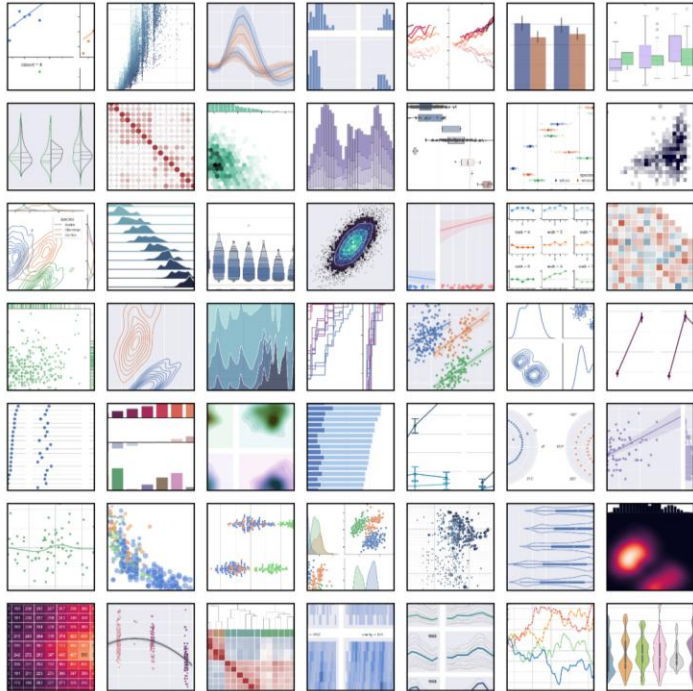
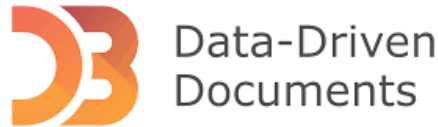


Image credit: <https://seaborn.pydata.org/examples/index.html>

Seaborn is a Python data visualization library that builds on top of Matplotlib, offering a high-level interface for drawing attractive and informative statistical graphics. It provides several benefits over Matplotlib, particularly when it comes to handling complex visualizations, statistical plots, and aesthetically appealing themes.

Since version 0.12, seaborn also offers the declarative `seaborn.objects` interface, e.g. `so.Plot(df, x='area', y='price').add(so.Dot())`.

Interactive Graphics & Web Applications: D3.js



D3.js (Data-Driven Documents) is a powerful JavaScript library used for creating dynamic, interactive, and data-driven visualizations in web browsers. It allows developers to bind data to the Document Object Model (DOM) and apply data-driven transformations to the document, enabling the creation of complex visual representations of data.

Interactive Graphics & Web Applications: d3blocks

Change Edge properties

Link threshold 0 840

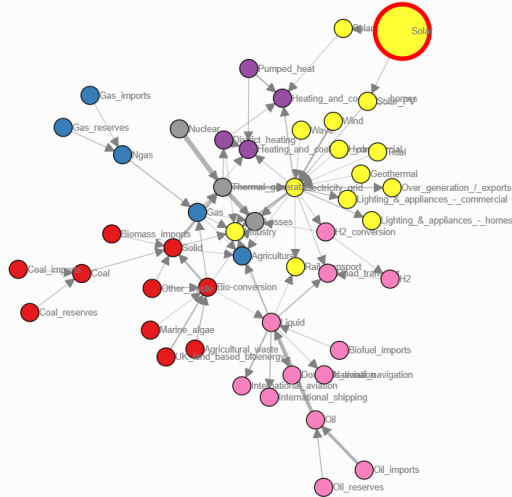


Image credit: <https://d3blocks.github.io/d3blocks/pages/html/d3graph.html>

d3blocks is a Python library designed to simplify the creation of interactive data visualizations by leveraging D3.js. It allows users to generate visualizations without directly writing D3.js code, making it accessible to those who may not be familiar with JavaScript or D3.js itself.

Interactive Graphics & Web Applications: Flask



Flask is a Python web application framework that lets you develop web applications easily. A web application framework represents a collection of libraries and modules that enable web application developers to write applications without worrying about low-level details such as protocol or thread management.

Interactive Graphics & Web Applications: Flask app

Flask app with HTML-template

```
app = Flask(__name__, template_folder='.', static_folder='.')

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':

        # Parse parameters from the URL query string
        obs = int(request.form['obs'])
        mean = float(request.form['mean'])
        stddev = float(request.form['stddev'])

        # Generate random data for the histogram
        data = np.random.normal(mean, stddev, obs)

        # Create histogram
        fig, ax = plt.subplots(figsize=(7, 5))
        ax.hist(data, bins=50, color="greenyellow", alpha=0.8)
        ax.set_xlabel('Value', fontsize=15)
        ax.set_ylabel('Frequency', fontsize=15)
        ax.set_title('Histogram', fontsize=15)
        ax.grid()

        # Save the plot to a .png image in memory
        img = io.BytesIO()
        fig.savefig(img, format='png')
        img.seek(0)

        # Plot url
        plot_url = base64.b64encode(img.getvalue()).decode()

        return render_template('index.html', plot_url=plot_url)
    else:
        return render_template('index.html')

if __name__ == '__main__':
    app.run()
```

Parameters to simulate data
obs, mean, stddev

Graphic (histogram)

image encoding

HTML template rendering

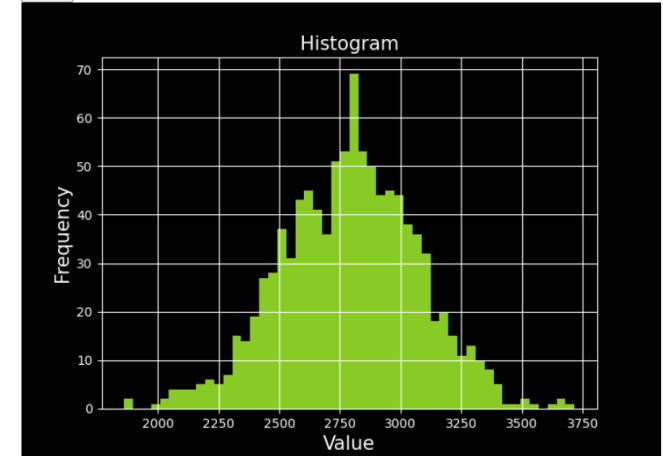


Histogram

Number of Observations:

Mean:

Standard Deviation:



Interactive Graphics & Web Applications: Flask HTML template

HTML template

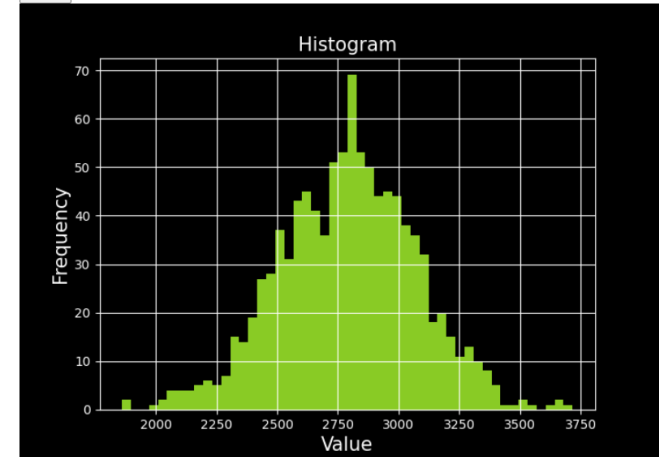
```
1 <!doctype html>
2 <html>
3   <head>
4     <title>Histogram</title>
5   </head>
6   <body>
7     <h1>Histogram</h1>
8     <form method="POST">
9       <label for="obs">Number of Observations:</label>
10      <input type="number" id="obs" name="obs" value="{{ obs }}" required><br><br>
11      <script>
12        document.addEventListener('DOMContentLoaded', function() {
13          const requiredFields = document.querySelectorAll('[required]');
14          for (let i = 0; i < requiredFields.length; i++) {
15            requiredFields[i].addEventListener('invalid', function() {
16              this.setCustomValidity('Please fill in this field');
17            });
18            requiredFields[i].addEventListener('input', function() {
19              this.setCustomValidity('');
20            });
21          }
22        });
23      </script>
24      <label for="mean">Mean:</label>
25      <input type="number" step="0.01" id="mean" name="mean" value="{{ mean }}" required><br><br>
26      <label for="stddev">Standard Deviation:</label>
27      <input type="number" step="0.01" id="stddev" name="stddev" value="{{ stddev }}" required><br><br>
28      <input type="submit" value="Submit">
29    </form>
30    {% if plot_url %}
31    
32    {% endif %}
33  </body>
34 </html>
```

Parameters:
obs
mean
stddev



Histogram

Number of Observations:
Mean:
Standard Deviation:



Interactive Graphics & Web Applications: FastAPI

main.py – histogram of real bike counts (Open Data Zürich)

```
import io
from typing import Literal
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from fastapi import FastAPI, Query
from fastapi.responses import Response

# Daily bike counts 2025 - Open Data Stadt Zürich (CC0)
df = pd.read_csv("velo_zuerich_2025.csv")
app = FastAPI(title="Zurich Bike Counts API")

@app.get("/histogram", response_class=Response,
        responses={200: {"content": {"image/png": {}}}})
def histogram(station: Literal["Mythenquai", "Hardbruecke",
                               "Langstrasse"] = "Mythenquai",
              bins: int = Query(30, ge=5, le=100)):
    """Histogram of daily bike counts at a counting station (2025)."""
    data = df.loc[df["station"] == station, "bikes"]
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.hist(data, bins=bins, color="#1769C4")
    ax.set(title=f"Daily bike counts {station} 2025",
           xlabel="Bikes per day", ylabel="Frequency (days)")
    buf = io.BytesIO()
    fig.savefig(buf, format="png")
    plt.close(fig)
    return Response(buf.getvalue(), media_type="image/png")
```

FastAPI is a modern Python web framework for building APIs. It is a popular alternative to Flask, especially for building APIs.

- Parameters are declared with **type hints** and validated automatically
- Interactive **API documentation** is generated at /docs
- **Asynchronous** (async/await) and fast
- No HTML template and no image encoding needed

Start the app:

```
pip install "fastapi[standard]"
fastapi dev main.py
```

Interactive Graphics & Web Applications: FastAPI app

Zurich Bike Counts API 0.1.0 OAS 3.1
/openapi.json

default

GET /**histogram** Histogram

Histogram of daily bike counts at a counting station (2025).

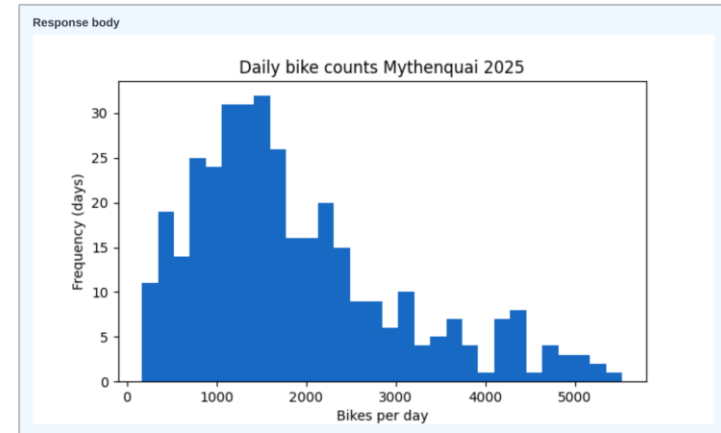
Parameters

Name	Description
station string (query)	Mythenquai
bins integer (query)	30

maximum: 100
minimum: 5

Execute Clear

Screenshots: Swagger UI of the example on the previous slide.
Data: Stadt Zürich Open Data, automatic bike counts 2025 (CC0).
Logo: fastapi.tiangolo.com



Left: station is a dropdown thanks to the Literal type hint.
Right: the returned PNG – clearly right-skewed.
Invalid input (station=Bahnhof) → HTTP 422.

Interactive Graphics & Web Applications: Plotly & Dash



Plotly is an open-source graphing library that allows users to create interactive, web-based visualizations. **Dash** is a low-code framework for building data apps in Python, R and Julia. Written on top of Plotly.js and React.js, Dash is used for building and deploying data apps with customized user interfaces.

Since Plotly 6 and Dash 3 (2025), Plotly Express works natively with pandas, Polars and PyArrow data frames (via Narwhals).

Interactive Graphics & Web Applications: Plotly example

Slices in volumetric data

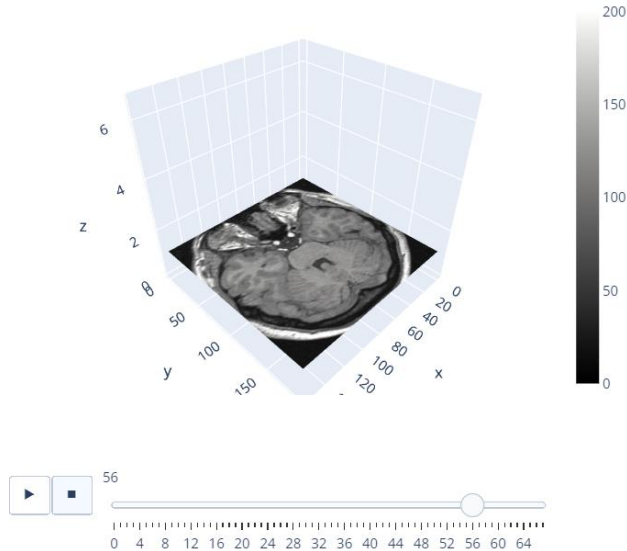


Image credit: <https://plotly.com/python/visualizing-mri-volume-slices/>

This **plotly** graphic shows an interactive 3D visualization of MRI slices from a volumetric dataset. The visualization includes controls for playing and pausing an animation that sequentially displays the slices, as well as a slider that lets users manually navigate through the slices. This interactive tool effectively represents the internal structure captured by the MRI, providing an intuitive way to explore the data.

Interactive Graphics & Web Applications: Dash example

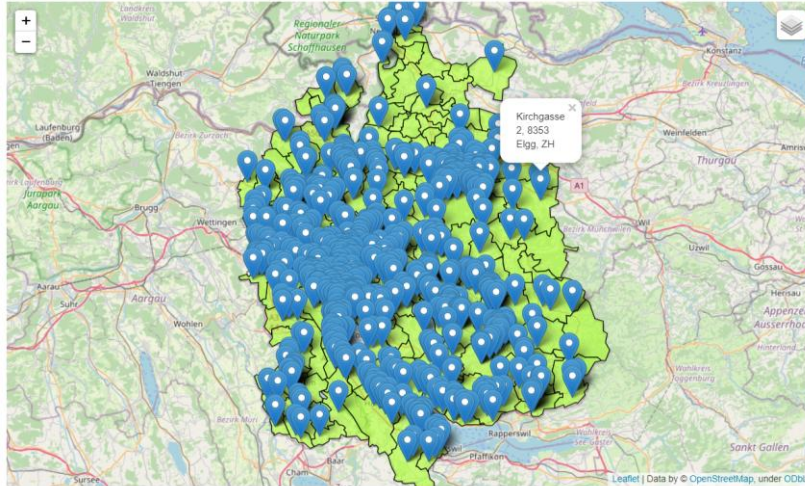


Dashboard showing over 4.5 million **Uber pickups** in New York City (April–September 2014) for different dates, times and locations built with Plotly and Dash.

[Dash Uber rides demo \(source code on GitHub\)](#)

Data: FiveThirtyEight (Uber TLC FOIL response), <https://github.com/fivethirtyeight/uber-tlc-foil-response>

Interactive Graphics & Web Applications: Leaflet / Folium



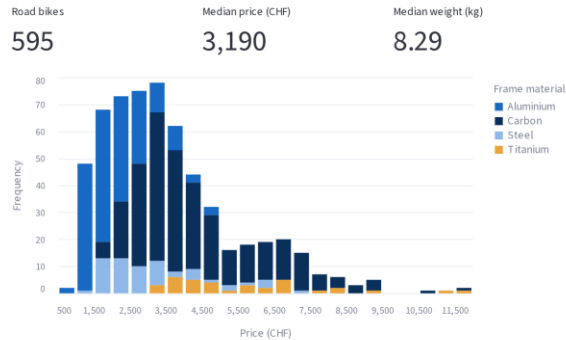
	web-scrapers-order	address_raw	lat	lon	bfs_number	bfs_name
0	1662023695-433	Sonnenbergstrasse 15, 8633 Wolfhausen, ZH	47.255714	8.804976	112	Bubikon
24	1662023720-634	Blumenbergstrasse 7, 8633 Wolfhausen, ZH	47.254879	8.793746	112	Bubikon
606	1662023745-834	8608 Bubikon, ZH	47.277386	8.800306	112	Bubikon
759	1662023701-503	8608 Bubikon, ZH	47.277386	8.800306	112	Bubikon
1	1662023745-820	Lavaterstr. 63, 8002 Zürich, ZH	47.361378	8.533339	261	Zürich

Leaflet is an open-source JavaScript library used for creating interactive maps. It is lightweight and designed to work efficiently across all major desktop and mobile platforms. Leaflet can be integrated with Python through libraries like folium, which allow Python users to create and display interactive Leaflet-based maps directly within Jupyter notebooks or web applications.

Interactive Graphics & Web Applications: Streamlit



Road bike explorer



Streamlit is an open-source Python framework that turns data scripts into interactive web apps with a few lines of code – no HTML, CSS or JavaScript required. Widgets such as sliders re-run the script automatically.

Similar frameworks: Dash, Shiny for Python, Panel, Gradio.

```
import streamlit as st
frames = st.sidebar.multiselect('Frame material', opts)
bins = st.sidebar.slider('Number of bins', 5, 60, 30)
st.altair_chart(price_hist(df, frames, bins))
# run: streamlit run app.py
```

Screenshot: example app (Streamlit + Altair) with synthetic road bike price data (code simplified).

Automated and AI-assisted EDA

Automated EDA reports (data profiling)

AI assistants for EDA: opportunities and pitfalls

Automated EDA: data profiling reports

Overview	Alerts 8	Reproduction
Alerts		
Dataset has 4 (0.6%) duplicate rows		Duplicates
area is highly overall correlated with price and 1 other fields		High correlation
luxurious is highly overall correlated with price_per_m2		High correlation
price is highly overall correlated with area and 1 other fields		High correlation
price_per_m2 is highly overall correlated with luxurious		High correlation
rooms is highly overall correlated with area and 1 other fields		High correlation
luxurious is highly imbalanced (58.9%)		Imbalance
area has 12 (1.7%) missing values		Missing

Profiling tools create a complete EDA report with one line of code: summary statistics, distributions, correlations, missing values, duplicates and automatic **alerts**.

Alternatives: Sweetviz, D-Tale.

```
from data_profiling import ProfileReport
report = ProfileReport(df)
report.to_file('report.html')
```

Screenshot: «Alerts» tab of a profiling report (synthetic apartment data).

AI-assisted EDA: opportunities and pitfalls

Opportunities

- Generate EDA code (pandas, seaborn, plotly) from natural-language prompts
- Suggest suitable statistics and plots for the data types at hand
- Explain results, error messages and unusual patterns
- Notebook agents (e.g. GitHub Copilot in VS Code/Codespaces, Jupyter AI, Colab Data Science Agent) can run whole EDA workflows

Pitfalls & good practice

- AI can hallucinate columns, functions or «findings» → verify every result
- Keep the generated code in the notebook to ensure reproducibility
- Data protection: do not upload personal or confidential data to external AI services
- AI accelerates EDA but does not replace your judgment – the objectives of EDA stay the same

Example prompt: «Load apartments.csv, report missing values and duplicates, and create histograms and a correlation heat map for all numeric variables with seaborn. Explain each step.»

Learning objectives

You ...

- can explain what exploratory data analysis (EDA) is and why it is useful
- can explain the different EDA methods and how they are classified
- can explain why summary statistics should always be complemented by graphics
- can name, explain and interpret measures of central tendency
- can name, explain and interpret measures of dispersion
- can name, explain and interpret skewness and kurtosis
- can explain the term contingency table
- can explain the terms correlation & covariance
- can name and explain univariate graphical EDA methods
- can name and explain multivariate graphical EDA methods
- know how to create graphics with matplotlib, seaborn, plotly/Dash, D3.js/d3blocks, folium, Flask/FastAPI and Streamlit
- can automate parts of the EDA process with profiling tools and AI assistants – and critically check the results

Literature: books and articles

Anscombe, F. J. (1973). Graphs in statistical analysis. *The American Statistician*, 27(1), 17–21.

Denis, D. J. (2021). *Applied univariate, bivariate, and multivariate statistics using Python*. John Wiley & Sons.

Matejka, J., & Fitzmaurice, G. (2017). Same stats, different graphs: Generating datasets with varied appearance and identical statistics through simulated annealing. *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, 1290–1294.

McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly. <https://wesmckinney.com/book/>

Navlani, A., Fandango, A., & Idris, I. (2021). *Python data analysis* (3rd ed.). Packt Publishing.

Rougier, N. P. (2021). *Scientific visualization: Python + Matplotlib*. <https://github.com/rougier/scientific-visualization-book>

Seltman, H. J. (2018). *Experimental design and analysis* (Chapter 4: Exploratory data analysis). Carnegie Mellon University.

Sundnes, J. (2020). *Introduction to scientific programming with Python*. Springer Nature.

Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.

VanderPlas, J. (2023). *Python data science handbook: Essential tools for working with data* (2nd ed.). O'Reilly.

Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.

Wilke, C. O. (2019). *Fundamentals of data visualization*. O'Reilly. <https://clauswilke.com/dataviz/>

Literature: online resources and documentation

fg-data-profiling (formerly ydata-profiling) (n.d.). <https://pypi.org/project/fg-data-profiling/>

FastAPI Documentation (n.d.). <https://fastapi.tiangolo.com>

Matplotlib User Guide (n.d.). <https://matplotlib.org/stable/users/index>

pandas User Guide (n.d.). https://pandas.pydata.org/docs/user_guide

Plotly Open Source Graphing Library for Python (n.d.). <https://plotly.com/python/>

Real Python Courses (n.d.). [Real Python course search](#)

seaborn User Guide and Tutorial (n.d.). <https://seaborn.pydata.org/tutorial.html>

Streamlit Documentation (n.d.). <https://docs.streamlit.io>